

SCRIPT-CONTROLLED ACCESS CONTROL LIST (ACL) SECURITY PLATFORM

Project Documentation & System Design Specifications

PROJECT TEAM DETAILS

Team Lead: Sumethaa P – Lead MERN Architect & Project Management

Team Members & Roles:

- **Sri Ranjana V:** Frontend Developer (React.js UI & Dashboard)
- **Bhuvana Lakshmi V:** Backend Developer (Node.js & Express.js REST API)
- **Deva Dharshini B:** Database Administrator (MongoDB & Mongoose Schemas)
- **Hema Sree K:** QA & Security Test Engineer

1. INTRODUCTION

1.1 Project Overview

The Script-Controlled Access Control List (ACL) Security Platform is a web-based security management system developed using the MERN Stack (MongoDB, Express.js, React.js, and Node.js). The platform enables organizations to dynamically control access to records based on field values, user roles, departments, and security policies.

Unlike traditional access control systems that rely on static permissions, this platform evaluates access permissions through configurable scripts and business rules, ensuring enhanced security, flexibility, and compliance.

1.2 Purpose

The purpose of this project is to:

- **Protect Sensitive Data:** Safeguard organizational resources against unauthorized access.
- **Prevent Breaches:** Restrict user interactions to strictly permitted records and actions.
- **Automate Decisions:** Execute real-time access evaluation driven by business rules.
- **Improve Auditability:** Maintain comprehensive logs for full compliance tracking.
- **Centralize Management:** Provide a unified administrative dashboard for security control.

2. IDEATION PHASE

2.1 Problem Statement

Organizations often face challenges in managing access permissions across multiple departments and applications. Traditional access control methods are rigid, difficult to maintain, and prone to security vulnerabilities.

There is a need for a dynamic security platform that can automatically determine access permissions based on record attributes and organizational policies.

2.2 Empathy Map Canvas

Users Think • Is my data secure? • Can I access only relevant information?	Users Feel • Concerned about unauthorized access. • Need confidence in data protection.
Users Say • Access control should be easy to manage. • Security should not affect productivity.	Users Do • Login to the system. • Access records. • Perform role-specific activities.
Pain Points • Complex permission management. • Security risks. • Manual access approvals.	Gains • Automated security decisions. • Improved user experience. • Better compliance.

2.3 Brainstorming

The team conducted structured brainstorming sessions and proposed key platform features:

- **Dynamic ACL Rule Engine:** Script-based policy engine for runtime permission evaluation.
- **Role-Based Access Control:** Granular role and permission assignment mapping.
- **Department Permissions:** Departmental boundary restrictions for multi-tenant data safety.
- **Security Dashboard:** Central UI for managing rules, users, and monitoring activity.
- **Audit Logging System:** Immutable tracking of access attempts, permissions, and violations.
- **Real-Time Evaluation:** Low-latency permission validation during API requests.
- **Security Analytics:** Visual reports on access trends, anomalies, and policy hits.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

- **Step 1: Login:** User enters credentials into the React frontend UI.
- **Step 2: Authentication:** System verifies user identity via JWT service.
- **Step 3: Authorization:** ACL engine evaluates permissions dynamically against active rules.
- **Step 4: Record Access:** User receives granted access or explicit denial based on result.
- **Step 5: Audit Logging:** All interaction attempts and security outcomes are logged automatically.

3.2 Solution Requirement

- **Functional Requirements:** User Registration, Authentication, Role Management, ACL Rule Management, Record Management, Access Decision Engine, Security Dashboard, Audit Logs.
- **Non-Functional Requirements:** Security, Scalability, Reliability, High Performance (low latency), System Maintainability.

3.3 Data Flow Diagram

System DFD Overview: Level 0 DFD:

User → Frontend Interface → Backend API → ACL Engine → Database

Level 1 DFD Steps:

- **1. User Request:** Initiation of resource access from client layer.
- **2. Auth Service:** Identity validation and JWT token decoding.
- **3. ACL Evaluation:** Rule engine execution with context metrics.
- **4. DB Validation:** Fetching resource data and rule definition schemas.
- **5. Access Decision:** Returning ALLOW/DENY authorization response.
- **6. Audit Logging:** Writing event metadata to immutable audit records.

3.4 Technology Stack

Category	Technologies / Tools Used
Frontend Layer	React.js, HTML5, CSS3, JavaScript (ES6+)
Backend Layer	Node.js, Express.js REST Framework
Database Layer	MongoDB, Mongoose ODM
Security & Tools	JWT Authentication, Password Hashing (Bcrypt), RBAC, VS Code, GitHub, Postman

4. PROJECT DESIGN

4.1 Problem Solution Fit

The proposed solution directly addresses security challenges by introducing dynamic access control mechanisms that evaluate permissions based on business logic and contextual field attributes rather than static, inflexible role assignments.

4.2 Proposed Solution

- **Secure Authentication:** Robust token-based user verification mechanism.
- **Dynamic ACL Rules:** Custom script and rule evaluation per request.
- **Record-Level Security:** Attribute and field-based visibility filtering.
- **Role-Based Authorization:** Multi-tiered role structure with dynamic overrides.
- **Real-Time Decisions:** Sub-millisecond permission checks integrated into API gateway.
- **Monitoring Dashboard:** Comprehensive portal for real-time security tracking.

4.3 Solution Architecture

Architectural Layer	Components & Responsibilities
Presentation Layer	React.js User Interface, Form Validation, State Management
Application Layer	Node.js & Express.js REST APIs, Request Routing
Security Layer	ACL Script Execution Engine, JWT Auth Middleware
Data Layer	MongoDB Collections, Mongoose Schemas & Indexing

Logging Layer	Audit Trail Collector, Security Event Monitoring Services
---------------	---

5. PROJECT PLANNING & SCHEDULING

Sprint Phase	Module Focus	Key Deliverables
Sprint 1	User Authentication	User Registration, Login API, JWT Security Integration
Sprint 2	ACL Rule Engine	Role Management, Dynamic Rule Evaluation Module
Sprint 3	Dashboard Development	Interactive User Dashboard, Security Analytics Portal
Sprint 4	Testing & Deployment	Functional, Performance & Security Testing, Production Build

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Functional Testing Matrix

Test Case	Expected Result
User Login	Successful Authentication & Token Generation
ACL Evaluation	Correct Access Decision Based on Active Rules
Record Access	Authorized Data Delivery / Proper Exclusion
Audit Logging	Complete Log Event Record Generated

6.2 Performance & Security Testing

- Performance Testing:** API Response Time Benchmarking, Concurrent User Load Simulations, Database Query Plan Optimization, High-Throughput Load Testing.
- Security Testing:** Authentication Flow Validation, Granular Authorization Boundary Checks, Session Hijacking Prevention, Comprehensive Vulnerability Assessments.

7. RESULTS

7.1 Output Deliverables

The following system key modules and outputs were successfully built and verified:

- Verified Module:** Login Page & Authentication Portal
- Verified Module:** Interactive User Dashboard
- Verified Module:** ACL Rule Configuration & Management Module
- Verified Module:** Record Management Screen with Dynamic Filtering

- **Verified Module:** Access Decision Results & Policy Evaluator Engine
- **Verified Module:** Audit Log Activity Dashboard
- **Verified Module:** Security Analytics & Threat Monitoring Visualizations

7.2 Result Summary

The platform successfully enforced access restrictions based on record field values and user roles while maintaining robust system security and low-latency performance.

8. ADVANTAGES & DISADVANTAGES

Advantages	Disadvantages
• Enhanced Data Security • Dynamic & Flexible Access Control • Reduced Administrative Maintenance • Improved Regulatory Compliance • Scalable MERN Stack Architecture • Immutable Audit Trail Support	• Initial Configuration Complexity • Slight Processing Overhead for Script Evaluation • Administrator Learning Curve for Custom Rule Writing

9. CONCLUSION

The Script-Controlled ACL Security Platform successfully demonstrates how dynamic access control mechanisms can significantly improve organizational security. By combining robust authentication, fine-grained authorization, and rule-based decision-making, the system provides a scalable, efficient, and secure solution for modern enterprise access management requirements.

10. FUTURE SCOPE

- **AI-Based Risk Analysis:** Integrating machine learning to analyze access patterns and identify potential security risks.
- **Behavioral Access Monitoring:** Tracking user behavior dynamically to adjust access privileges in real-time.
- **Multi-Factor Authentication (MFA):** Adding extra security layers like OTPs and biometric verification.
- **Cloud Deployment & Scaling:** Optimizing architecture for cloud environments like AWS and Azure.
- **Predictive Security Analytics:** Leveraging predictive models to foresee vulnerabilities and breach risks.
- **Real-Time Threat Detection:** Instant alert triggers and automated lockdown upon anomaly detection.
- **ML Access Recommendations:** Suggesting optimal permission rules based on historical user roles and usage.

11. APPENDIX

Resource / Artifact	Details / Link Container
Source Code Tech Stack	MERN Stack (MongoDB, Express.js, React.js, Node.js)
Dataset Link	http://example.com/dataset-link (Insert dataset or sample records link)

GitHub Repository	http://github.com/example/script-acl-platform (Insert repository URL)
--------------------------	---